

Customer churn - Data Insights

2. Data Loading and Understanding

```
# Load the csv data to a pandas dataframe
df = pd.read_csv("Telecom_Customer_Churn")
```

```
df.shape
```

```
(7043, 21)
```

```
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	No	Yes	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	No	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	No	Yes	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	No	No	No

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   customerID          7043 non-null   object  
 1   gender              7043 non-null   object  
 2   SeniorCitizen       7043 non-null   int64   
 3   Partner             7043 non-null   object  
 4   Dependents          7043 non-null   object  
 5   tenure              7043 non-null   int64   
 6   PhoneService        7043 non-null   object  
 7   MultipleLines       7043 non-null   object  
 8   InternetService     7043 non-null   object  
 9   OnlineSecurity      7043 non-null   object  
10  OnlineBackup        7043 non-null   object  
11  DeviceProtection    7043 non-null   object  
12  TechSupport         7043 non-null   object  
13  StreamingTV         7043 non-null   object  
14  StreamingMovies     7043 non-null   object  
15  Contract            7043 non-null   object  
16  PaperlessBilling    7043 non-null   object  
17  PaymentMethod       7043 non-null   object  
18  MonthlyCharges      7043 non-null   float64  
19  TotalCharges        7043 non-null   object  
20  Churn               7043 non-null   object  
dtypes: float64(1), int64(2), object(18)
```

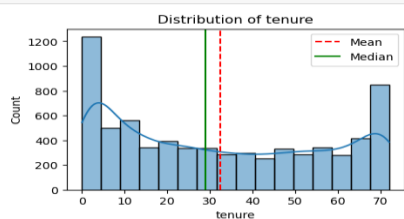
```
df.head(10)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport
0	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	No	No
1	Male	0	No	No	34	Yes	No	DSL	Yes	No	Yes	Yes
2	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	No	No
3	Male	0	No	No	45	No	No phone service	DSL	Yes	No	Yes	Yes
4	Female	0	No	No	2	Yes	No	Fiber optic	No	No	No	No
5	Female	0	No	No	8	Yes	Yes	Fiber optic	No	No	Yes	Yes
6	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No	Yes	No	No
7	Female	0	No	No	10	No	No phone service	DSL	Yes	No	No	No
8	Female	0	Yes	No	28	Yes	Yes	Fiber optic	No	No	Yes	Yes
9	Male	0	No	Yes	62	Yes	No	DSL	Yes	Yes	No	No

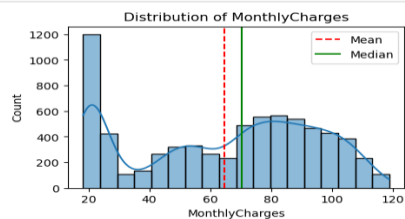
```
df[df["TotalCharges"]==" "]
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport
488	Female	0	Yes	Yes	0	No	No phone service	DSL	Yes	No	Yes	Yes
753	Male	0	No	Yes	0	Yes	No	No	No internet service	No internet service	No internet service	No internet service
936	Female	0	Yes	Yes	0	Yes	No	DSL	Yes	Yes	Yes	Yes
1082	Male	0	Yes	Yes	0	Yes	Yes	No	No internet service	No internet service	No internet service	No internet service
1340	Female	0	Yes	Yes	0	No	No phone service	DSL	Yes	Yes	Yes	Yes
3331	Male	0	Yes	Yes	0	Yes	No	No	No internet service	No internet service	No internet service	No internet service
3826	Male	0	Yes	Yes	0	Yes	Yes	No	No internet service	No internet service	No internet service	No internet service
4380	Female	0	Yes	Yes	0	Yes	No	No	No internet service	No internet service	No internet service	No internet service
5218	Male	0	Yes	Yes	0	Yes	No	No	No internet service	No internet service	No internet service	No internet service
6670	Female	0	Yes	Yes	0	Yes	Yes	DSL	No	Yes	Yes	Yes
6754	Male	0	No	Yes	0	Yes	Yes	DSL	Yes	Yes	No	No

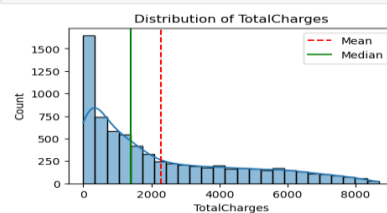
```
plot_histogram(df, "tenure")
```



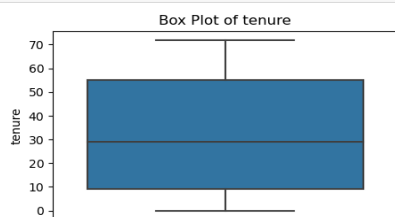
```
plot_histogram(df, "MonthlyCharges")
```



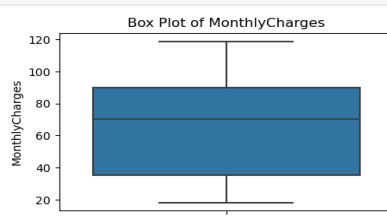
```
plot_histogram(df, "TotalCharges")
```



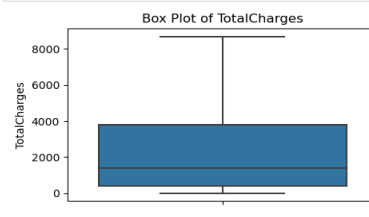
```
plot_boxplot(df, "tenure")
```



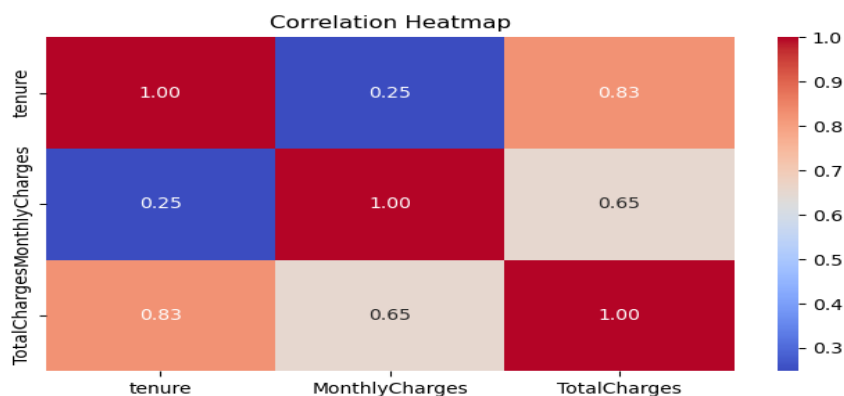
```
plot_boxplot(df, "MonthlyCharges")
```

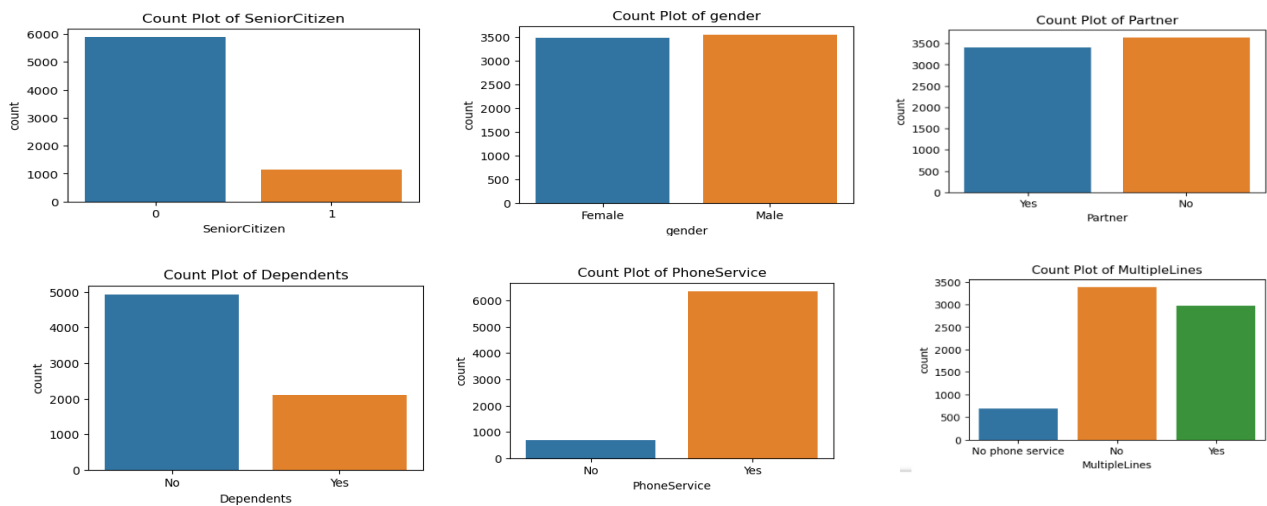


```
plot_boxplot(df, "TotalCharges")
```



```
# correlation matrix - heatmap
plt.figure(figsize=(8, 4))
sns.heatmap(df[["tenure", "MonthlyCharges", "TotalCharges"]].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```





Training with default hyperparameters

```
# dictionary of models
models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "XGBoost": XGBClassifier(random_state=42)
}

# dictionary to store the cross validation results
cv_scores = {}

# perform 5-fold cross validation for each model
for model_name, model in models.items():
    print(f"Training {model_name} with default parameters")
    scores = cross_val_score(model, X_train_smote, y_train_smote, cv=5, scoring="accuracy")
    cv_scores[model_name] = scores
    print(f"{model_name} cross-validation accuracy: {np.mean(scores):.2f}")
    print("-"*70)
```

Training Decision Tree with default parameters
Decision Tree cross-validation accuracy: 0.78

Training Random Forest with default parameters
Random Forest cross-validation accuracy: 0.84

Training XGBoost with default parameters
XGBoost cross-validation accuracy: 0.83

cv_scores

```
{'Decision Tree': array([0.69202899, 0.70574018, 0.82537764, 0.83806647, 0.84350453]),
 'Random Forest': array([0.73067633, 0.77039275, 0.90392749, 0.89969789, 0.90030211]),
 'XGBoost': array([0.71799517, 0.75649547, 0.89728097, 0.89003021, 0.91057402])}
```

Random Forest gives the highest accuracy compared to other models with default parameters

```
rfc = RandomForestClassifier(random_state=42)
```

```
rfc.fit(X_train_smote, y_train_smote)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

6. Model Evaluation

```
# evaluate on test data
y_test_pred = rfc.predict(X_test)

print("Accuracy Score:\n", accuracy_score(y_test, y_test_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_test_pred))
print("Classification Report:\n", classification_report(y_test, y_test_pred))
```

```
Accuracy Score:
0.7785663591199432
Confusion Matrix:
[[878 158]
 [154 219]]
Classification Report:
precision    recall  f1-score   support

     0       0.85     0.85     0.85     1036
     1       0.58     0.59     0.58     373

 accuracy
macro avg       0.72     0.72     0.72     1409
weighted avg       0.78     0.78     0.78     1409
```

Final Step — Save Best Model

```
.]: # save best model

final_model = {
    "model": best_model,
    "feature_names": X.columns.tolist()
}

with open("final_customer_churn_model.pkl", "wb") as f:
    pickle.dump(final_model, f)

print("Final model saved successfully!")
```

Final model saved successfully!